process-adapter

Process Adapter

The process adapter provides an interface for managing long-running processes such as servers, services, and background workers. It is typically used to start services that integration tests depend on.

ProcessAdapter Interface

Defined in `adapter_process.go`:

```
type ProcessAdapter interface {
    Launch(ctx context.Context, config ProcessConfig) error
    IsRunning() bool
    WaitForReady(ctx context.Context, timeout time.Duration) error
    Stop() error
}
```

Method Summary

Method	Purpose
`Launch`	Start the process with the given configuration
`IsRunning`	Check if the process is currently alive
`WaitForReady`	Block until the process is ready or timeout expires
`Stop`	Terminate the running process gracefully

Note that `IsRunning` and `Stop` do not take a `context.Context` parameter. Process state checking and termination are local operations that do not benefit from context cancellation.

Configuration Type

ProcessConfig

```
type ProcessConfig struct {
    Command string            `json:"command"`
    Args    []string          `json:"args"`
    WorkDir string            `json:"work_dir"`
    Env     map[string]string `json:"env"`
}
```

- `Command` is the executable path or name (resolved via PATH).
- `Args` are command-line arguments.
- `WorkDir` is the working directory. If empty, the current directory is used.
- `Env` provides additional environment variables. These are appended to the current process environment (`os.Environ()`), so existing variables are inherited.

ProcessCLIAdapter

The built-in implementation manages a subprocess via `os/exec`. It supports graceful shutdown with a SIGTERM-then-SIGKILL sequence and readiness polling.

Constructor

```
adapter := userflow.NewProcessCLIAdapter()
```

No arguments are required. The process configuration is provided at launch time.

Launch

```
err := adapter.Launch(ctx, userflow.ProcessConfig{
    Command: "/usr/bin/my-server",
    Args:    []string{"--port=8080", "--debug"},
    WorkDir: "/opt/app",
    Env: map[string]string{
        "DB_HOST": "localhost",
        "DB_PORT": "5432",
    },
})
```

The adapter:

1. Checks that no process is already running. Returns an error if one is.
2. Creates the command with `exec.CommandContext`.

3. Sets the working directory and environment.
4. Starts the process with `cmd.Start()`.
5. Spawns a goroutine that calls `cmd.Wait()` and closes a `done` channel when the process exits.

IsRunning

Uses a two-phase check:

1. Checks the `done` channel (non-blocking select). If closed, the process has exited.
2. Sends `signal(0)` to the process as a liveness probe.

Both checks are protected by a mutex.

WaitForReady

Polls `IsRunning()` every 200ms until the process is detected as running or the timeout expires:

```
err := adapter.WaitForReady(ctx, 10*time.Second)
```

Note: This checks process liveness, not application-level readiness. For HTTP readiness checks, combine with `APIAdapter.Available()` or use the `TestEnvironment` health checking.

Stop

Performs a graceful shutdown sequence:

1. Sends `SIGTERM` to the process.
2. Waits up to 5 seconds for the process to exit.
3. If still running after 5 seconds, sends `SIGKILL`.
4. Waits for the process goroutine to complete.

```
err := adapter.Stop()
```

If the process has already exited, both SIGTERM and SIGKILL return nil without error.

Thread Safety

All methods are protected by a `sync.Mutex`. The adapter is safe for concurrent use, though typical usage is sequential (launch, wait, use, stop).

Example: Starting a Server for Integration Tests

```
proc := userflow.NewProcessCLIAdapter()

// Start the server
err := proc.Launch(ctx, userflow.ProcessConfig{
    Command: "./bin/server",
    Args:    []string{"--port=9090"},
    WorkDir: "/path/to/project",
    Env: map[string]string{
        "GIN_MODE": "test",
    },
})
if err != nil {
    return err
}

// Wait for it to be running
err = proc.WaitForReady(ctx, 15*time.Second)
if err != nil {
    return err
}

// ... run API tests against localhost:9090 ...

// Clean up
err = proc.Stop()
```

Typical Usage with Environment Challenges

The `ProcessAdapter` is commonly used inside `EnvironmentSetupChallenge` and `EnvironmentTeardownChallenge` to manage service lifecycle:

```
proc := userflow.NewProcessCLIAdapter()

setup := userflow.NewEnvironmentSetupChallenge(
    "CH-ENV-001",
    func(ctx context.Context) error {
        err := proc.Launch(ctx, userflow.ProcessConfig{
            Command: "./bin/server",
        })
        if err != nil {
            return err
        }
        return proc.WaitForReady(ctx, 30*time.Second)
    },
```

```
    60*time.Second,
)

teardown := userflow.NewEnvironmentTeardownChallenge(
    "CH-ENV-TEARDOWN",
    func(ctx context.Context) error {
        return proc.Stop()
    },
)
```